

MUSEU DAS AMAZÔNIAS 2026

EGP PMO — DIRETORIA DE PROJETOS

PROCESSO DE ATUALIZAÇÃO DO DASHBOARD PMO

Procedimento Operacional Padrão (POP)

Responsável pelo processo	EGP PMO — Diretoria de Projetos
Versão	v1.0
Data de emissão	26 de maio de 2026
Próxima revisão	Novembro de 2026 ou após mudança estrutural
Ferramenta principal	Claude (Cowork + Claude Code)
Status	Ativo

Sumário

Sumário	2
1. Finalidade.....	4
2. Escopo	4
Incluso	4
Excluído	4
3. Papéis e Responsabilidades (RACI).....	5
4. Pré-requisitos e Ferramentas	5
4.1 Software a instalar (única vez)	5
Verificação das instalações	6
4.2 Acesso ao repositório	6
4.3 API Key do Google Sheets.....	6
5. Fluxo do Processo	7
Passo 0 — Puxar a versão mais recente	7
Passo 1 — Subir o servidor local	7
Passo 2 — Fazer as alterações	7
Regra crítica de edição	7
Validação de JavaScript obrigatória	7
Passo 3 — Testar localmente	8
Desktop.....	8
Mobile	8
Celular físico (quando disponível)	8
Passo 4 — Publicar	8
Regras de commit.....	9
6. Skills Automatizadas	10
6.1 doc-sync — Sincronização de Documentação	10
O que faz	10
Como instalar.....	10
Como acionar.....	10
Quando usar	10
Fluxo de aprovação	10
Documentação completa	10
6.2 code-audit — Auditoria de Código	11
O que faz	11
Como instalar.....	11
Modos de uso	11
7. Fontes de Dados.....	12
7.1 Planilhas Google Sheets	12

7.2 Índices de colunas (referência técnica).....	12
Cronograma — função _parseWBS	12
Requisições — função _parseREQS.....	12
7.3 Regras de auto-status	13
Ranking de pior status (rollup hierárquico)	13
8. Exceções e Armadilhas Conhecidas	14
9. Procedimento de Rollback	15
Opção A — Reverter um commit específico (recomendado).....	15
Opção B — Recuperar versão antiga de um arquivo	15
Opção C — Voltar todo o repositório a uma versão (CUIDADO)	15
Via interface do GitHub	15
10. Métricas de Processo.....	16
11. Documentos Relacionados	17
12. Histórico de Revisões	17

1. Finalidade

Este Procedimento Operacional Padrão (POP) descreve, de forma completa e reproduzível, o processo de atualização do Dashboard PMO do Museu das Amazônias 2026 (MAZ ELD). O objetivo é garantir que qualquer membro habilitado da equipe EGP PMO consiga realizar manutenções, publicar alterações e utilizar as ferramentas automatizadas (skills) sem depender de conhecimento tácito ou suporte externo.

2. Escopo

Incluso

- Configuração inicial do ambiente de trabalho (primeira vez)
- Workflow de edição, teste local e publicação no GitHub Pages
- Uso das skills automatizadas: doc-sync e code-audit
- Atualização e versionamento da documentação técnica
- Procedimento de rollback em caso de erro em produção

Excluído

- Alterações diretas nas planilhas do Google Sheets (Cronograma e Requisições)
- Criação de novas funcionalidades de grande porte sem aprovação do responsável
- Mudanças na API Key ou IDs de planilha sem autorização da Diretoria
- Administração do Google Cloud Platform (GCP)

3. Papéis e Responsabilidades (RACI)

Legenda: R = Responsável (executa) · A = Accountable (responde pelo resultado) · C = Consultado · I = Informado

Etapa do Processo	Responsável	Accountable	Consultado	Informado
Clonar repositório e configurar ambiente	Responsável	Accountable	-	PMO
Solicitar acesso ao GitHub como colaborador	Responsável	Accountable	-	-
Receber/guardar API Key do Google Sheets	Responsável	Accountable	-	Diretoria
Rodar servidor local (SERVE_DASHBOARD.bat)	Responsável	Accountable	-	-
Editar index.html / mobile.html via Claude Code	Responsável	Accountable	Claude Code	PMO
Validar JS com node --check	Responsável	Accountable	Claude Code	-
Testar no browser desktop (localhost:8000)	Responsável	Accountable	-	-
Testar no celular (via IP local)	Responsável	Accountable	-	-
Executar git commit e git push	Responsável	Accountable	-	PMO
Confirmar publicação no GitHub Pages	Responsável	Accountable	-	Diretoria
Rodar skill doc-sync após mudanças relevantes	Responsável	Accountable	Cowork	PMO
Aprovar atualizações de documentação (doc-sync)	Responsável	Accountable	Cowork	-
Mover versões antigas para Manual/old_versions/	Responsável	Accountable	-	-
Reverter commit em caso de erro em produção	Responsável	Accountable	Claude Code	PMO
Atualizar CLAUDE.md e ONBOARDING.md	Responsável	Accountable	-	PMO

4. Pré-requisitos e Ferramentas

4.1 Software a instalar (única vez)

Todas as instalações são feitas apenas uma vez, na configuração inicial do ambiente. Após isso o ambiente fica pronto para qualquer ciclo de atualização.

Ferramenta	Função	Como é usada no processo
Git	Controle de versão e publicação	Clonar repo, commitar e publicar alterações no GitHub Pages
Python 3.x	Servidor local de preview	Roda SERVE_DASHBOARD.bat (python -m http.server 8000)
Node.js	Validação de JavaScript	node --check arquivo.js — detecta erros de sintaxe JS antes do commit
Claude Code	Edição de arquivos e git	Edita index.html e mobile.html; roda comandos bash; faz commit/push
Claude Cowork	Skills especializados	Roda skill doc-sync (sincronização de docs) e cria documentos Word/PDF
Navegador web	Testes locais	Hard refresh (Ctrl+Shift+R) para testar desktop e mobile localmente
Google Sheets	Fonte de dados	Cronograma e Requisições — planilhas com compartilhamento público ativado
GitHub Pages	Publicação automática	Serve index.html e mobile.html em pmo-creator.github.io/maz-dashboard/
VS Code (opcional)	Editor alternativo	Pode ser usado para edições simples; Claude Code é preferido

Verificação das instalações

Após instalar, abrir o terminal e confirmar:

```
git --version
python --version
node --version
```

4.2 Acesso ao repositório

- Solicitar ao responsável atual que adicione você como colaborador em:
 - github.com/PMO-creator/maz-dashboard → Settings → Collaborators → Add people
- Clonar o repositório (somente na primeira vez):

```
git clone https://github.com/PMO-creator/maz-dashboard
```

4.3 API Key do Google Sheets

- A chave de API é necessária para o dashboard buscar dados em tempo real.
- Nunca alterar a API Key ou os IDs das planilhas sem confirmar com o responsável.
- Em caso de chave expirada ou inválida, o indicador mostrará . Solicitar nova chave em: console.cloud.google.com → APIs e serviços → Credenciais.
- Projeto GCP de referência: maz-dashboard-495414 · Restrição: pmo-creator.github.io/*

5. Fluxo do Processo

O fluxo de atualização segue sempre a mesma sequência de 5 passos. Nunca pular etapas — a validação local protege a produção.

GIT PULL → SERVIDOR LOCAL → EDITAR → TESTAR → COMMIT / PUSH

Passo 0 — Puxar a versão mais recente

Antes de qualquer edição, garantir que a cópia local está sincronizada com o repositório remoto:

```
# Abrir terminal na pasta maz-dashboard  
git pull
```

Se houver conflitos, resolvê-los antes de prosseguir. Em caso de dúvida, solicitar apoio ao responsável.

Passo 1 — Subir o servidor local

O servidor local permite visualizar o dashboard sem afetar a produção. Dar duplo-clique no arquivo:

```
SERVE_DASHBOARD.bat
```

Ou, alternativamente, no terminal:

```
python -m http.server 8000
```

O browser abrirá automaticamente em `http://localhost:8000`. Manter o servidor aberto durante toda a sessão de edição.

Passo 2 — Fazer as alterações

Abrir o Claude Code na pasta maz-dashboard e descrever a alteração desejada. O Claude Code editará os arquivos HTML diretamente.

Regra crítica de edição

- Edições em `index.html` e `mobile.html` devem ser feitas via `Python str.replace()` no bash — NUNCA com o Edit tool do Claude Code. Arquivos grandes com template literals quebram ao ser editados pelo Edit tool.
- Toda alteração que afeta layout, KPIs, lógica de dados ou parsing de colunas normalmente precisa ser replicada em ambos os arquivos (`index.html` E `mobile.html`). Verificar sempre os dois.

Validação de JavaScript obrigatória

Após qualquer alteração que envolva código JavaScript:

1. Extrair o bloco `<script>` do HTML para um arquivo temporário:

```
# No terminal (exemplo para index.html)  
python -c "import re;  
open('temp.js', 'w').write(re.search(r'<script>(.*?)</script>',  
open('index.html').read(), re.DOTALL).group(1))"
```

2. Rodar a validação:

```
node --check temp.js
```

3. Deletar o arquivo temporário após o teste:

```
del temp.js
```

Passo 3 — Testar localmente

Testar SEMPRE em desktop e em visualização mobile antes de publicar. Desde 01/07/2026 o mobile.html foi removido — o dashboard é um único index.html responsivo; o teste "mobile" é feito redimensionando a janela ou usando o modo dispositivo do DevTools no mesmo arquivo:

Desktop

- Abrir: `http://localhost:8000/index.html`
- Fazer hard refresh: `Ctrl+Shift+R`
- Confirmar indicador  Ao vivo no canto superior direito
- Testar as funcionalidades afetadas pela mudança
- Verificar filtros: Status, Eixo, Data, Responsável

Mobile

- Abrir: `http://localhost:8000/index.html` (mesmo arquivo do desktop — não existe mais mobile.html)
- Abrir o DevTools (F12), ativar o modo dispositivo (`Ctrl+Shift+M` no Chrome) e simular um celular (ex.: iPhone SE, Galaxy S20)
- Fazer hard refresh: `Ctrl+Shift+R`
- Testar as abas: Gantt, Status Report, Requisições, Áreas e Diretoria

Celular físico (quando disponível)

- Computador e celular devem estar na mesma rede Wi-Fi.
- Descobrir o IP do computador: rodar `ipconfig` no terminal e anotar o Endereço IPv4.
- No celular, acessar: `http://[SEU-IP]:8000/index.html` — o redirect automático abre o mobile.html.
- Atenção: o IP pode mudar quando a rede muda. Verificar com `ipconfig` antes de cada teste.

Indicador (canto superior direito)	Significado
 Ao vivo · HH:MM	Tudo funcionando. Dados do Google Sheets foram carregados com sucesso.
 Cronograma OK · REQ erro	Planilha de cronograma OK, mas a planilha de Requisições falhou. Verificar compartilhamento da aba de REQs.
 Erro — dados locais	Ambos os fetches falharam. Dashboard exibe último cache. Verificar conectividade e configuração da API Key.

Passo 4 — Publicar

Publicar SOMENTE após aprovação visual no teste local. A publicação é irreversível de forma instantânea (rollback leva ~2 minutos).

```
# Adicionar apenas os arquivos alterados
git add index.html mobile.html

# Commit com descrição clara em português
```



```
git commit -m "Corrigir: [descrição objetiva do que foi alterado e por quê]"  
  
# Publicar  
git push
```

Aguardar aproximadamente 2 minutos para o GitHub Pages atualizar. Confirmar fazendo hard refresh em pmo-creator.github.io/maz-dashboard.

Regras de commit

- Sempre descrever o commit em português com o que foi alterado e por quê.
- Um commit por alteração — não acumular múltiplas mudanças num só commit.
- NUNCA usar `git push --force` na branch main.
- NUNCA editar arquivos diretamente pelo GitHub (vai direto para produção sem teste).

6. Skills Automatizadas

As skills são módulos especializados do Claude que automatizam tarefas repetitivas. Elas ficam instaladas no Cowork e no Claude Code e são acionadas por comandos de texto.

6.1 doc-sync — Sincronização de Documentação

O que faz

Compara o index.html atual com o snapshot da última execução, identifica mudanças funcionais relevantes e atualiza automaticamente os documentos afetados:

- Manual de Uso e Manutenção
- Guia de Onboarding
- Ficha Técnica
- ONBOARDING.md (arquivo técnico do repositório)

Como instalar

4. Abrir o Cowork com a pasta maz-dashboard montada.
5. Abrir o arquivo: doc-sync/doc-sync.skill
6. Clicar em Save skill no Cowork.
7. A skill ficará disponível em sessões futuras.

Como acionar

No chat do Cowork (com maz-dashboard montado), digitar qualquer um dos gatilhos:

- doc-sync
- atualizar docs
- documentação
- sincronizar manuais

Quando usar

Sempre após um git push que contenha mudanças funcionais relevantes: novos filtros, novos KPIs, novas abas, mudanças de layout significativas, correções de lógica de parsing ou qualquer alteração que impacte a experiência do usuário.

Fluxo de aprovação

A skill propõe as atualizações antes de gravar. O responsável revisa e aprova (ou rejeita) cada alteração antes que qualquer arquivo seja sobrescrito. Documentação final salva em Manual/ com a versão antiga movida para Manual/old_versions/.

Documentação completa

Ver: maz-dashboard/doc-sync/SKILL.md

6.2 code-audit — Auditoria de Código

O que faz

Analisa index.html e mobile.html em busca de problemas de segurança, qualidade de código, armadilhas JavaScript e dependências externas.

Como instalar

8. Abrir o Claude Code na pasta maz-dashboard.
9. Instalar o arquivo: code-audit/code-audit.skill

Modos de uso

Comando	Modo	Quando usar
audita o que mudou	Diff apenas	Verificação rápida antes de um commit
auditoria completa	Arquivos inteiros	Revisão periódica ou antes de release importante
audita o index	Só index.html	Alteração afetou apenas o desktop
audita o mobile	Só mobile.html	Alteração afetou apenas o mobile

7. Fontes de Dados

7.1 Planilhas Google Sheets

Planilha	ID da Planilha	Aba de Referência
Cronograma	17nttJ_ShqWztvDWH3I59iNqboLqkviZs3_PM5J3ihdA	master data
Requisições	1azrdS4OGO-CWD1ods69i8iZJcwq4oyISdT2n_tu1uJM	Planilha de Status de Compras Prod

Requisito crítico: ambas as planilhas precisam ter o compartilhamento configurado como 'Qualquer pessoa com o link pode ver'. Sem isso o dashboard retorna erro 403/400 e o indicador fica vermelho.

7.2 Índices de colunas (referência técnica)

Cronograma — função _parseWBS

Coluna	Índice (0-based)	Campo no dashboard
B	1	Eixo
C	2	Grupo
D	3	Marco
E	4	Tarefa
G	6	Responsável
H	7	Status
I	8	Data de início
J	9	Data de término

Requisições — função _parseREQS

Coluna	Índice (0-based)	Campo no dashboard
A	0	Nº Requisição
B	1	Comprador
D	3	Prioridade (usar APENAS esta coluna)
E	4	Descrição
F	5	Status
G	6	Fornecedor
M	12	Finalização do Serviço

Índice 10 (coluna K) — “Chegada do Material” (nova em Jul/2026). Complementa o índice 12 (“Finalização do Serviço”) — a antiga coluna única de previsão virou duas datas distintas.

7.3 Regras de auto-status

O dashboard calcula e sobrescreve o status automaticamente com base nas regras abaixo. Estas regras executam a cada carregamento de dados e a cada 15 minutos (auto-refresh):

Condição na planilha	Status resultante no dashboard
Subtask sem status + sem data	Definir datas
A iniciar + sem data de início	Definir datas
Grupo/eixo sem status definido	Definir datas
Qualquer status + data fim já passou	Atrasado
Qualquer status + data fim nos próximos 7 dias	Risco de atraso
Feito / Cancelado / Cancelado-Congelado	Nunca muda (imune às regras acima)

Ranking de pior status (rollup hierárquico)

Marco = pior status das tarefas filhas. Grupo = pior status dos marcos. Eixo = pior status dos grupos.

Atrasado > Risco de atraso > Em andamento > Definir datas > A iniciar > Feito > Cancelado/Congelado

8. Exceções e Armadilhas Conhecidas

A tabela abaixo reúne os problemas mais comuns e como resolvê-los. Consultar antes de escalar para o responsável.

Situação / Armadilha	O que fazer
Dashboard branco sem erro no console	Verificar: (a) null bytes no HTML, (b) palavra function ausente, (c) JS truncado sem <code></script></code> , (d) template literals aninhados. Rodar <code>node --check</code> no bloco script extraído para <code>.js</code> .
Template literals aninhados (crases dentro de <code>\${}\$</code>)	NUNCA usar crase dentro de <code>\${}\$</code> dentro de outro crase — <code>node --check</code> passa mas o browser quebra. Reescrever usando concatenação ou variáveis intermediárias.
Indicador vermelho (Erro — dados locais)	Não é bug do dashboard. Verificar: (a) compartilhamento da planilha ('Qualquer pessoa com o link pode ver'), (b) API Key ativa e com restrição correta em <code>console.cloud.google.com</code> .
Edit tool do Claude Code falha em <code>index.html</code>	Arquivo contém backticks JS (template literals). Usar <code>Python str.replace()</code> no bash para editar. Nunca usar o Edit tool em arquivos HTML com template literals.
String não encontrada no Replace (Python)	Arquivo usa CRLF (Windows). Normalizar antes: <code>content.replace('\r\n', '\n')</code> e depois do replace converter de volta se necessário.
KPIs zerados nas Requisições	Índices de coluna incorretos. Confirmar que Prioridade usa APENAS coluna D (índice 3). Coluna B (índice 1) gera falsos positivos. Ver commit 7662590.
Mudança afetou só <code>index.html</code> mas <code>mobile</code> está errado	Verificar se a mudança de layout, KPI ou parsing precisa ser replicada em <code>mobile.html</code> . Regra: alterações em lógica de dados sempre afetam os dois arquivos.
GitHub Pages não atualizou após push	Aguardar ~2 minutos. Fazer hard refresh no browser (<code>Ctrl+Shift+R</code>). Verificar Actions do repo para ver se o deploy foi executado.
<code>node --check</code> não aceita <code>.html</code> (Node v22+)	Extraír apenas o bloco <code><script></code> para arquivo <code>.js</code> temporário e rodar <code>node --check</code> arquivo.js. Deletar o arquivo temporário após o teste.
Necessidade de rollback urgente em produção	Ver seção 9 — Rollback. Preferir <code>git revert</code> (mantém histórico). Nunca usar <code>git push --force</code> sem avisar o responsável.
Aba Diretoria: status não muda sozinho mesmo com data vencida	Não é bug. A função <code>preprocessStatusesDiretoria</code> (aba Diretoria, nova em Jul/2026) usa o status vindo direto da planilha e não sobrescreve por data vencida, diferente de <code>preprocessStatuses</code> (aba principal), que marca Atrasado/Risco de atraso automaticamente. Ao alterar a lógica de status, verificar sempre se a mudança deve valer para as duas cópias da árvore: WBS (principal) e WBS_DIR (Diretoria).
Exportar PDF do modal Comparativo (aba Diretoria) não usa o wizard padrão	Comportamento esperado. O botão “Exportar PDF” do modal Comparativo (aba Diretoria, <code>buildComparativoModal</code>) chama <code>window.print()</code> diretamente, diferente do mecanismo padrão <code>_runExportPDF</code> usado no resto do dashboard (wizard de exportação). Se o print não funcionar como esperado, verificar configuração de impressão do navegador, não a lógica do dashboard.

9. Procedimento de Rollback

Se uma alteração causar problemas em produção, seguir imediatamente uma das opções abaixo. Avisar o responsável antes de iniciar o rollback.

Opção A — Reverter um commit específico (recomendado)

Cria um novo commit que desfaz as mudanças. O histórico fica intacto — esta é a abordagem mais segura.

```
# Ver histórico de commits
git log --oneline

# Reverter o commit problemático (substituir HASH pelo hash real)
git revert HASH
git push
```

Opção B — Recuperar versão antiga de um arquivo

Útil para comparar ou copiar trechos da versão anterior sem fazer rollback total.

```
# Ver como o arquivo estava num commit anterior
git show HASH:index.html > versao_antiga.html
```

Opção C — Voltar todo o repositório a uma versão (CUIDADO)

ATENÇÃO: esta opção apaga tudo feito após o commit indicado. Usar apenas em último caso e sempre avisar o responsável.

```
git reset --hard HASH
git push --force
```

Via interface do GitHub

10. Acessar: github.com/PMO-creator/maz-dashboard
11. Clicar na aba Commits.
12. Encontrar o commit desejado → clicar em <> (Browse files).
13. Baixar o arquivo da versão antiga manualmente.

10. Métricas de Processo

Métrica	Meta	Como medir
Tempo entre alteração e publicação em produção	< 30 minutos	Timestamp do commit vs. timestamp da solicitação
Taxa de rollbacks por ciclo de atualização	< 5%	Commits de revert / total de commits de feature
Cobertura de teste local antes do push	100% dos pushes	Checklist preenchido a cada ciclo
Documentação sincronizada (doc-sync)	Rodar em 100% dos pushes com mudança funcional	Presença de relatório em doc-sync/reports/ após cada push relevante
Tempo de disponibilidade do dashboard (indicador 🕒)	> 95% do horário comercial	Inspeção visual diária ou monitoramento via Google Sheets

11. Documentos Relacionados

Documento	Localização	Finalidade
ONBOARDING.md	maz-dashboard/ONBOARDING.md	Guia técnico completo para desenvolvedores — leitura obrigatória antes da primeira edição
CLAUDE.md	maz-dashboard/CLAUDE.md	Instruções específicas para o Claude Code e Cowork ao trabalhar no projeto
SKILL.md (doc-sync)	maz-dashboard/doc-sync/SKILL.md	Documentação completa da skill de sincronização de documentação
Manual de Uso e Manutenção	maz-dashboard/Manual/	Manual para usuários finais e gestores do dashboard
Guia de Onboarding	maz-dashboard/Manual/	Guia de onboarding técnico versionado em docx e pdf
Ficha Técnica	maz-dashboard/Manual/	Especificações técnicas do dashboard (arquitetura, APIs, colunas)
Dashboard público	pmo-creator.github.io/maz-dashboard/	URL de produção — confirmar após cada push
Repositório GitHub	github.com/PMO-creator/maz-dashboard	Código-fonte, histórico de commits e configuração do GitHub Pages

12. Histórico de Revisões

Versão	Data	Autor	Descrição
v1.0	26/05/2026	EGP PMO / Claude (Cowork)	Emissão inicial do POP — processo de atualização do Dashboard MAZ 2026
v2.0	Jul/2026	EGP PMO / Claude (Cowork)	Aba Diretoria adicionada (Gantt/EAP/N2/Comparativo); mobile.html removido — dashboard agora único e responsivo; nova coluna REQS “Chegada do Material”.